Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)
[Course Code: 23ECSPE1AD]

Submitted by

R POORVI 1BF24EC115

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

INDEX

ACKNOWLEDGEMENT 3

INTRODUCTION TO EDA PLAYGROUND 4

CODE AND SIMULATION WITH WAVEFORMS 4

1. Arithmetic Logic Unit 4
2. Comparator 6
3. 4-Bit Parity Generator and Checker 7
4. 4-Bit Barrel Shifter 8
5. Timer Module 10
6. Ring Counter 11
7. Johnson Counter 12
8. Synchronous and Asynchronous counter 13
9. Shift Register 16
10. Divide-by-N-Counter 18
11. Mealy and Moore FSMs 19
12. Random Sequence Generator 23

Conclusion 25

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Professor Ajay Devarapalli** for his continuous support, guidance, and valuable feedback throughout the duration of this Alternate Assessment Test. His clear guidance helped me stay on track and learn from this experience.

I am also thankful to the **Electronics and Communications** Department and the faculty at **BMS College of Engineering** for providing the necessary resources and facilities to complete this work successfully.

Lastly, I would like to extend my thanks to my classmates for their helpful discussions, and to my family for their constant encouragement while I was working on this assignment.

INTRODUCTION TO EDA PLAYGROUND:

EDA Playground is a web-based electronic design automation (EDA) platform used for the simulation and synthesis of digital circuits.

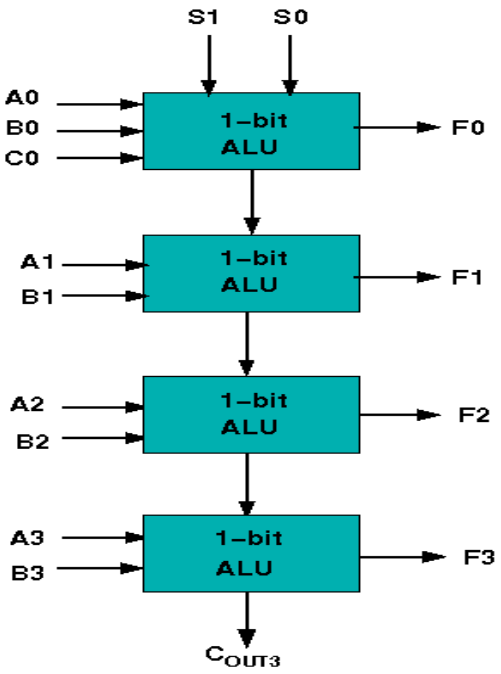
In the context of Advanced Logic Design, it serves as the primary tool for designing and verifying complex hardware circuits using Hardware Description Languages (HDLs) such as Verilog and VHDL. By providing cloud-based access to industry-standard simulators and an integrated waveform viewer (EPWave), it eliminates the need for heavy local software installations, allowing for seamless coding, testbench execution, and timing analysis of digital designs directly from a web browser.

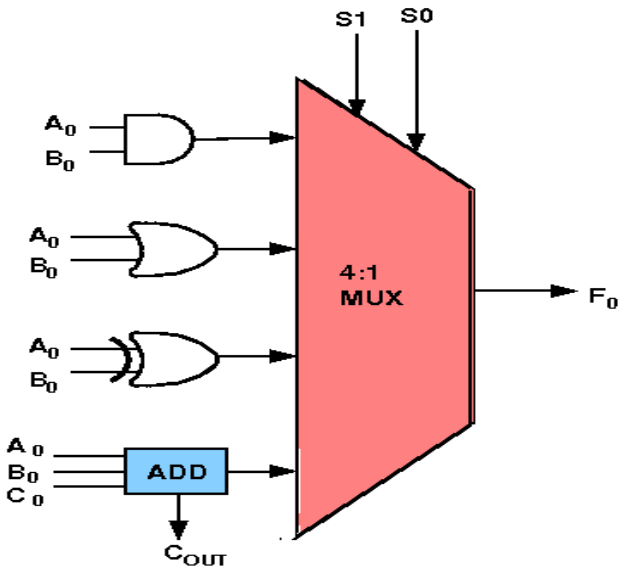
CODE AND SIMULATION WITH WAVEFORMS:

1. **Arithmetic Logic Unit:**

   An ALU is the mathematical brain of a computer processor. It is a digital circuit that performs basic arithmetic (like addition and subtraction) and logical operations (like AND, OR) on binary numbers. Based on a control signal, it decides which specific operation to perform on the given inputs. It then outputs the result, making it the core component of any CPU.

Block Diagram:





Code:

testbench.sv

```
module tb_ALU;
  reg [3:0] A, B;
  reg [1:0] Sel;
  wire [3:0] Out;

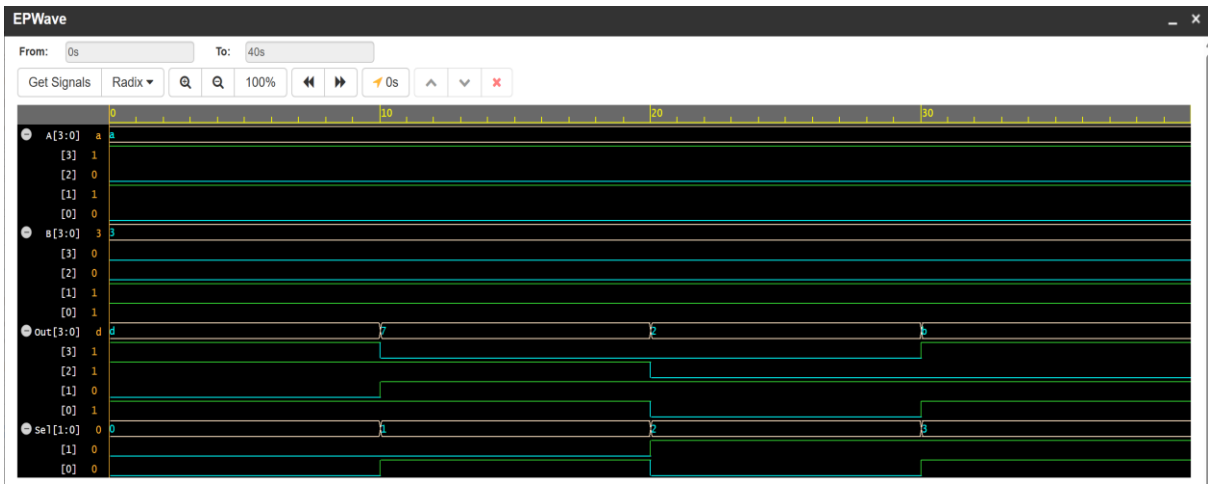
  ALU uut (.A(A), .B(B), .Sel(Sel), .Out(Out));

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_ALU);
    A = 4'b1010; B = 4'b0011;
    Sel = 2'b00; #10; // Add
    Sel = 2'b01; #10; // Sub
    Sel = 2'b10; #10; // AND
    Sel = 2'b11; #10; // OR
    $finish;
  end
endmodule
```

design.sv

```
module ALU(input [3:0] A, B, input [1:0] Sel, output reg [3:0] Out);
  always @(*) begin
    if (Sel == 2'b00) Out = A + B;       // Add
    else if (Sel == 2'b01) Out = A - B;  // Subtract
    else if (Sel == 2'b10) Out = A & B;  // AND
    else Out = A | B;                    // OR
  end
endmodule
```

Waveform:

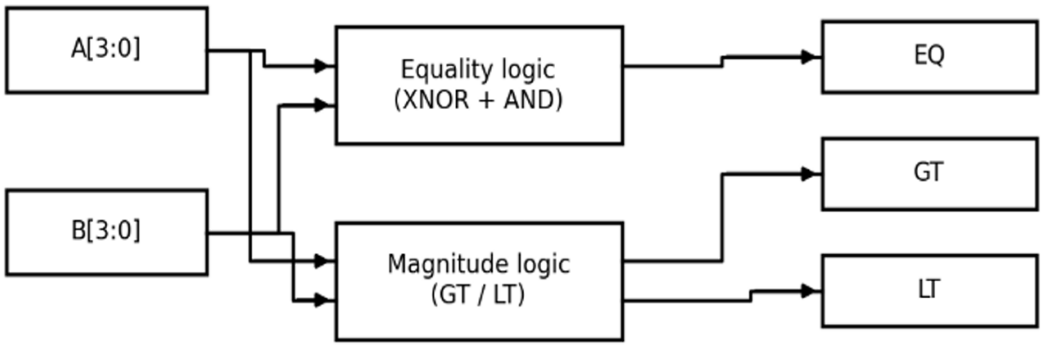


The waveform shows the input operands A and B along with the select lines that determine the operation. When the select lines change, the ALU performs the corresponding arithmetic or logical operation. The output Y displays the calculated result, while the carry output indicates a carry generated during addition. The waveform verifies that the ALU performs the selected operations correctly.

2. Comparator:

A comparator is a logic circuit used to compare the size or magnitude of two binary numbers. It takes two numbers (like A and B) as inputs and determines their relationship to one another. It typically provides three distinct outputs indicating whether A is greater than B, equal to B, or less than B. It is widely used in sorting devices, password matching, and digital thermostats.

Block Diagram:



Code:

testbench.sv

```
module tb_Comparator;
  reg [3:0] A, B;
  wire Eq, Gt, Lt;

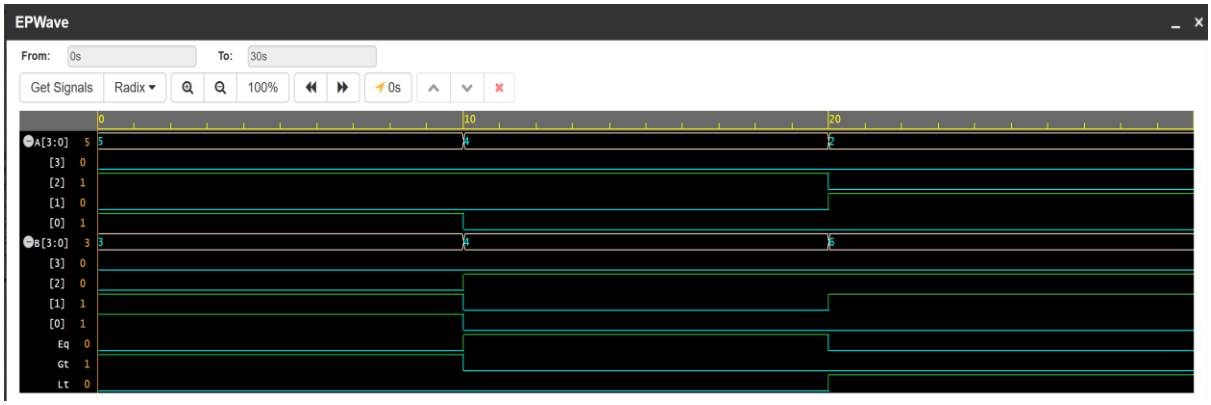
  Comparator uut (.A(A), .B(B), .Eq(Eq), .Gt(Gt), .Lt(Lt));

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_Comparator);
    A = 5; B = 3; #10; // A > B
    A = 4; B = 4; #10; // A == B
    A = 2; B = 6; #10; // A < B
    $finish;
  end
endmodule
```

design.sv

```
module Comparator(input [3:0] A, B, output Eq, Gt, Lt);
  assign Eq = (A == B); // Equal
  assign Gt = (A > B);  // Greater than
  assign Lt = (A < B);  // Less than
endmodule
```

Waveform:

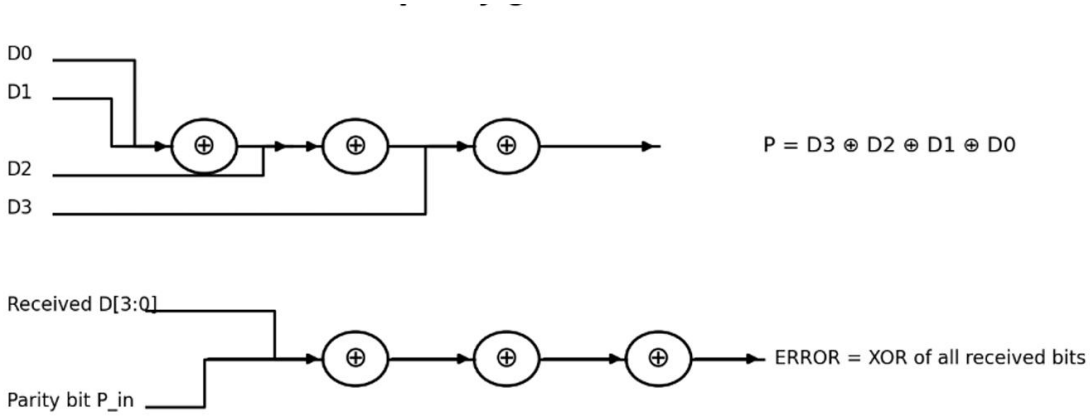


The waveform displays the two input numbers A and B and the corresponding GT, EQ and LT outputs. When A is greater than B, the GT output becomes HIGH. When both inputs are equal, EQ becomes HIGH, and when A is less than B, LT becomes HIGH. The waveform verifies that the comparator correctly identifies the relationship between the two binary numbers.

3. 4-Bit Parity Generator and Checker:

A parity generator produces a parity bit using XOR operations to help detect errors in transmitted data. A parity checker verifies the received data and parity bit to identify parity errors.

Block Diagram:



Code:

testbench.sv

```
module tb_Parity;
  reg [3:0] Data;
  wire Even_Parity;

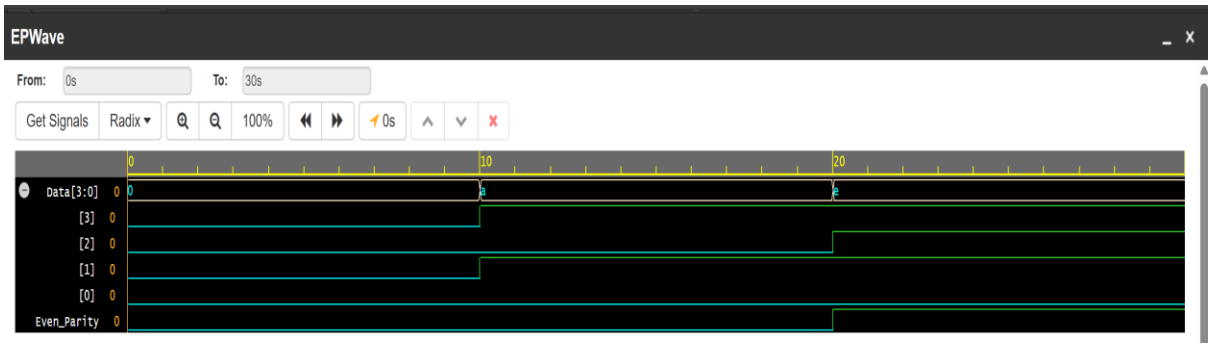
  ParityGenerator uut (.Data(Data), .Even_Parity(Even_Parity));

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_Parity);
    Data = 4'b0000; #10;
    Data = 4'b1010; #10; // Two 1s -> Parity = 0
    Data = 4'b1110; #10; // Three 1s -> Parity = 1
    $finish;
  end
endmodule
```

design.sv

```
module ParityGenerator(input [3:0] Data, output Even_Parity);
  assign Even_Parity = Data[3] ^ Data[2] ^ Data[1] ^ Data[0];
endmodule
```

Waveform:

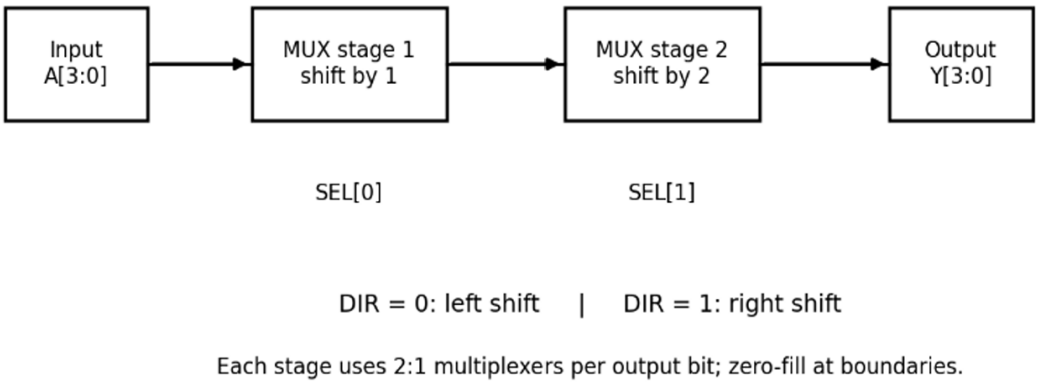


The waveform shows the four-bit input data, generated parity bit and error output. The parity generator calculates the parity bit using XOR operations on the input data. The error output becomes HIGH when the received data and parity bit produce an odd number of ones, indicating a parity error. The waveform demonstrates the generation of parity and detection of errors in the received data.

4. **4-Bit Barrel Shifter:**

   A barrel shifter is a digital circuit that can shift a binary number left or right by multiple positions in a single, instant clock cycle. Unlike a standard shift register that moves data slowly one bit at a time, a barrel shifter uses hardware switches (multiplexers) to shift data by any specified amount immediately. This makes it incredibly fast and useful in processors for quick multiplication or division.

Block Diagram:



Code:

testbench.sv

```
module tb_BarrelShifter;
  reg [3:0] In;
  reg [1:0] Shift;
  wire [3:0] Out;

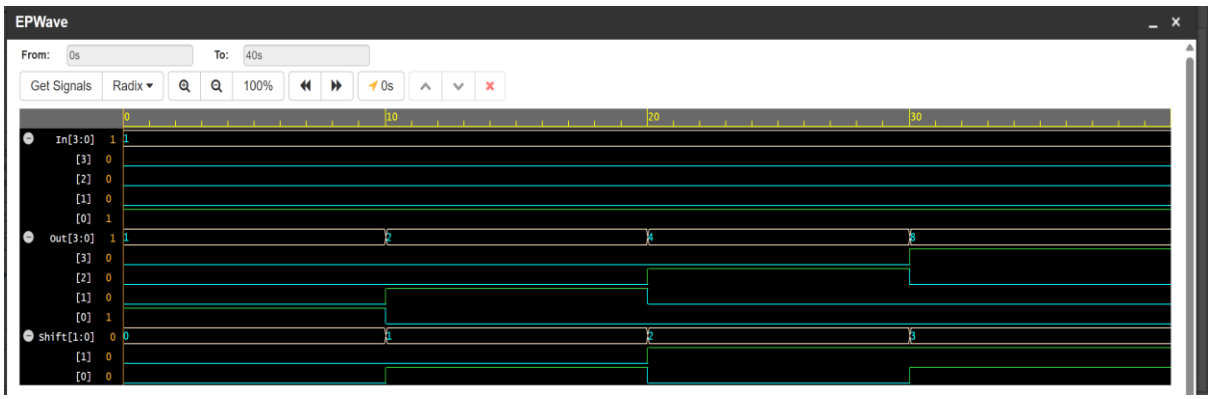
  BarrelShifter uut (.In(In), .Shift(Shift), .Out(Out));

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_BarrelShifter);
    In = 4'b0001;
    Shift = 2'b00; #10; // No shift
    Shift = 2'b01; #10; // Shift by 1
    Shift = 2'b10; #10; // Shift by 2
    Shift = 2'b11; #10; // Shift by 3
    $finish;
  end
endmodule
```

design.sv

```
module BarrelShifter(input [3:0] In, input [1:0] Shift, output reg
[3:0] Out);
  always @(*) begin
    Out = In << Shift;
  end
endmodule
```

Waveform:

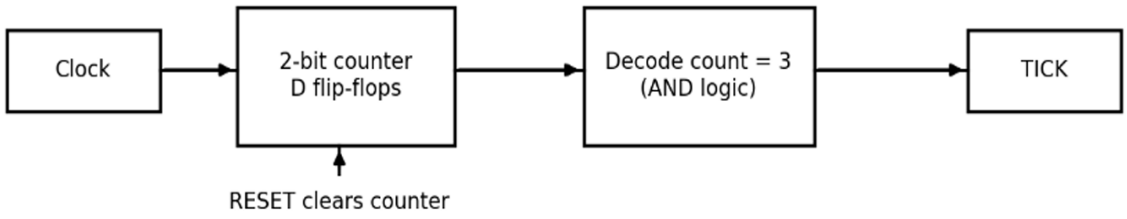


The waveform displays the input data, shift amount, direction control and output data. When the direction is set to left, the input bits shift towards the left by the specified number of positions. When the direction is set to right, the bits shift towards the right, with zeros filling the vacant positions. The waveform verifies that the barrel shifter performs the required shifting operation according to the control inputs.

5. **Timer Module:**

   A timer module uses a clock-driven counter to measure a fixed number of clock cycles. It generates a tick pulse whenever the counter reaches its specified terminal count.

Block Diagram:



Code:

testbench.sv

```
module tb_Timer;
  reg clk, reset;
  wire [3:0] count;

  Timer uut (.clk(clk), .reset(reset), .count(count));

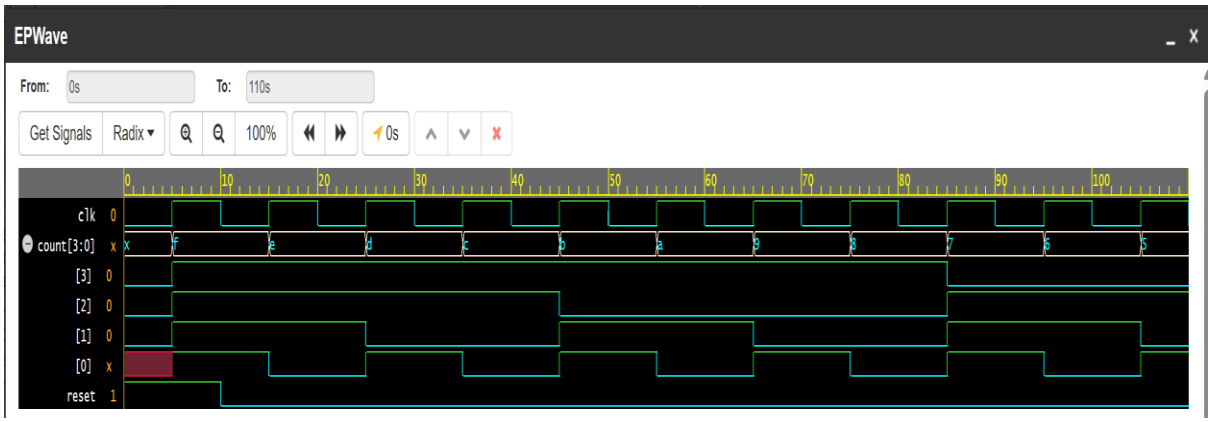
  always #5 clk = ~clk; // Generate Clock

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_Timer);
    clk = 0; reset = 1; #10;
    reset = 0; #100; // Let it count down
    $finish;
  end
endmodule
```

design.sv

```
module Timer(input clk, reset, output reg [3:0] count);
  always @(posedge clk) begin
    if (reset) count = 4'b1111;
    else count = count - 1;
  end
endmodule
```

Waveform:

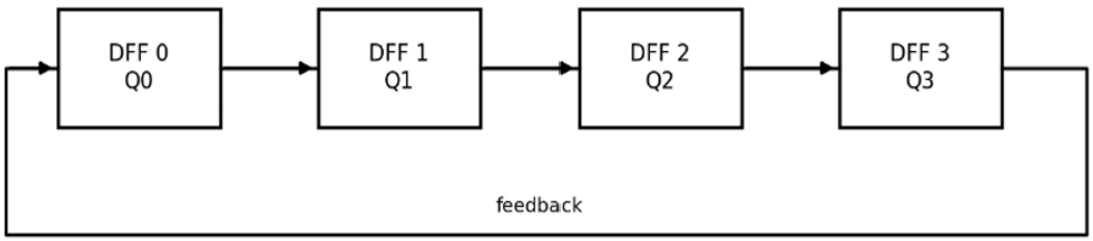


The waveform shows the clock, reset and tick output signals of the timer circuit. Initially, the reset signal clears the counter and keeps the tick output LOW. After reset is deactivated, the counter increments on every rising edge of the clock until it reaches the terminal count. The tick output becomes HIGH for one clock cycle every four-clock cycle, demonstrating the correct operation of the timer.

6. **Ring Counter:**

   A ring counter is a special shift register where the output of the last memory cell (flip-flop) is wired directly back to the first one, forming a loop. Only one flip-flop in the ring holds a '1' at any time, while all others hold '0'. As the clock ticks, this single '1' passes from one flip-flop to the next in a continuous circle, like a baton in a relay race. It is often used to control the step-by-step sequencing of operations in a machine.

Block Diagram:



RESET initializes the state; outputs are Q[3:0].

Code:

testbench.sv

```
module tb_RingCounter;
  reg clk, reset;
  wire [3:0] Q;

  RingCounter uut (.clk(clk), .reset(reset), .Q(Q));

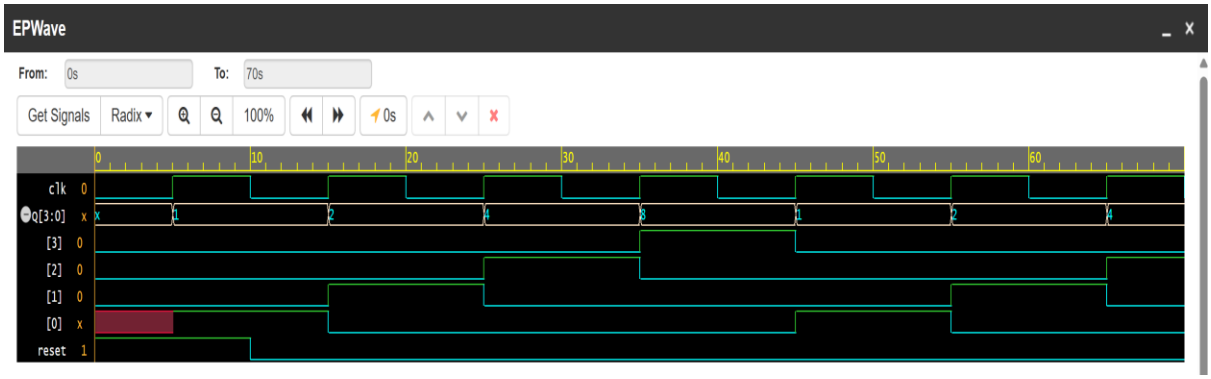
  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_RingCounter);
    clk = 0; reset = 1; #10;
    reset = 0; #60;
    $finish;
  end
endmodule
```

design.sv

```
module RingCounter(input clk, reset, output reg [3:0] Q);
  always @(posedge clk) begin
    if (reset) Q = 4'b0001;
    else Q = {Q[2:0], Q[3]};
  end
endmodule
```

Waveform:

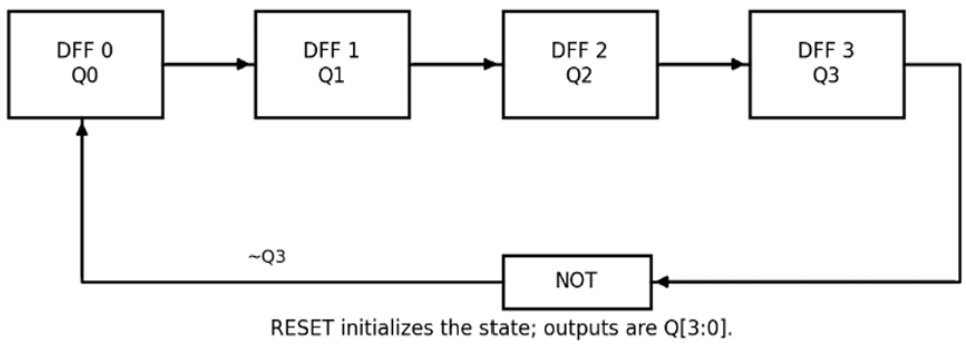


The waveform displays the clock, reset and four-bit counter output. Initially, reset loads the counter with state 0001, in which only one bit is HIGH. On every rising clock edge, this HIGH bit shifts to the next position and circulates through the four flip-flops. The waveform shows the repeating sequence 0001, 0010, 0100 and 1000, verifying the operation of the ring counter.

7. Johnson Counter:

Also known as a twisted ring counter, this is like a regular ring counter, but the inverted output of the last flip-flop is fed back into the first one. This slight twist means the register fills up with 1s and then empties out with 0s in a continuous sequence (e.g., 0000, 1000, 1100, 1110, 1111, 0111...). It is highly efficient because it provides twice as many distinct states as a standard ring counter using the same amount of hardware.

Block Diagram:



Code:

testbench.sv

```
module tb_Johnson;
  reg clk, reset;
  wire [3:0] Q;

  JohnsonCounter uut (.clk(clk), .reset(reset), .Q(Q));

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_Johnson);
    clk = 0; reset = 1; #10;
    reset = 0; #100;
    $finish;
  end
endmodule
```

design.sv

```
module JohnsonCounter(input clk, reset, output reg [3:0] Q);
  always @(posedge clk) begin
    if (reset) Q = 4'b0000;
    else Q = {Q[2:0], ~Q[3]};
  end
endmodule
```

Waveform:



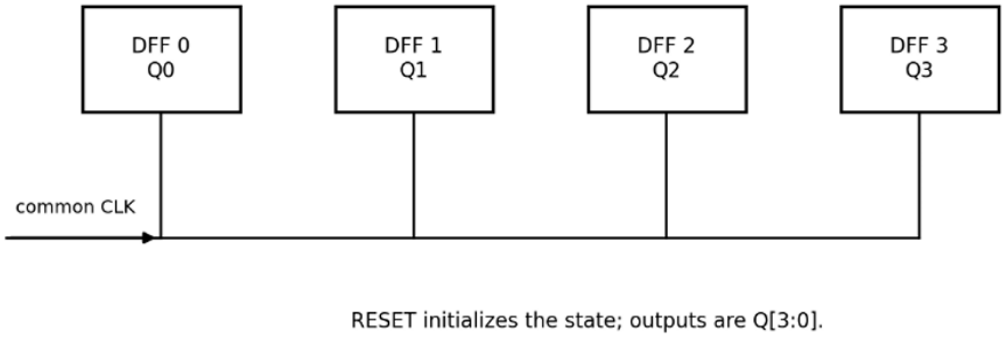
The waveform shows the clock, reset and four-bit output of the Johnson counter. Initially, reset clears all flip-flops, producing the state 0000. At every rising clock edge, the bits shift and the inverted output of the last flip-flop feeds the first stage. The waveform displays eight repeating states, verifying the correct operation of the four-bit Johnson counter.

8. Synchronous and Asynchronous counter

Synchronous Counter:

A synchronous counter consists of flip-flops driven by the same clock signal. A four-bit synchronous up counter increments its binary value on every rising clock edge.

Block Diagram:



RESET initializes the state; outputs are Q[3:0].

Code:

testbench.sv

```
module tb_SyncCounter;
  reg clk, reset;
  wire [3:0] Q;

  SyncCounter uut (.clk(clk), .reset(reset), .Q(Q));

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_SyncCounter);
    clk = 0; reset = 1; #10;
    reset = 0; #100;
    $finish;
  end
endmodule
```

design.sv

```
module SyncCounter(input clk, reset, output reg [3:0] Q);
  always @(posedge clk) begin
    if (reset) Q = 4'b0000;
    else Q = Q + 1;
  end
endmodule
```

Waveform:

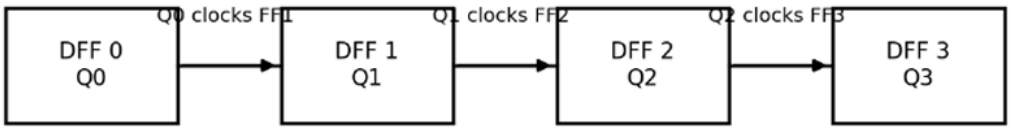


The waveform displays the clock, reset and four-bit output of the synchronous counter. Initially, reset sets the counter output to 0000. After reset is deactivated, all flip-flops respond to the same rising edge of the clock, and the binary count increases by one at each clock pulse. The counter progresses from 0000 to 1111 and then returns to 0000, demonstrating synchronous counting.

Asynchronous Counter:

An asynchronous counter, also called a ripple counter, uses the output of one flip-flop to clock the next flip-flop. The binary count advances through the states as transitions ripple through the stages.

Block diagram:



RESET initializes the state; outputs are Q[3:0].

Code for Asynchronous Counter:

testbench.sv

```
module tb_AsyncCounter;
  reg clk, reset;
  wire [3:0] Q;

  AsyncCounter uut (.clk(clk), .reset(reset), .Q(Q));
  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_AsyncCounter);

    clk = 0;
    reset = 1; #10; // Hold reset high for 10 units

    reset = 0; #200; // Let it count to see the ripple effect

    $finish;
  end
endmodule
```

design.sv

```

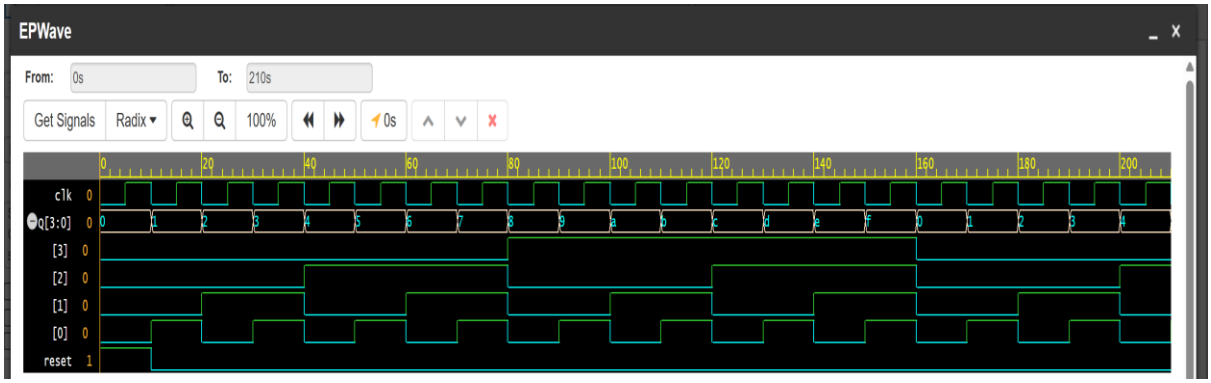
  // Bit 1: Toggles on the falling edge of Bit 0 (Ripple effect)
  always @(negedge q0 or posedge reset) begin
    if (reset) q1 <= 0;
    else q1 <= ~q1;
  end

  // Bit 2: Toggles on the falling edge of Bit 1
  always @(negedge q1 or posedge reset) begin
    if (reset) q2 <= 0;
    else q2 <= ~q2;
  end

  // Bit 3: Toggles on the falling edge of Bit 2
  always @(negedge q2 or posedge reset) begin
    if (reset) q3 <= 0;
    else q3 <= ~q3;
  end

  // Combine the individual bits into the 4-bit output
  assign Q = {q3, q2, q1, q0};
endmodule
```

Waveform:

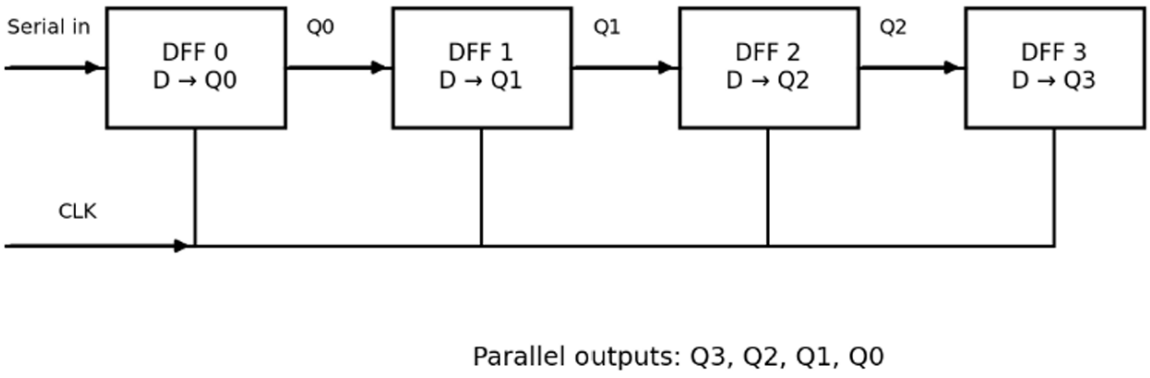


The waveform shows the clock, reset and four-bit output of the asynchronous ripple counter. Initially, reset clears all the flip-flops and sets the output to 0000. When clock pulses are applied, the first flip-flop toggles, and its output triggers the subsequent flip-flops to produce the binary count. The waveform demonstrates the ripple-counting operation, with small propagation delays between successive stages.

9. Shift Register:

A shift register stores binary data using a series of flip-flops and shifts the data on each active clock edge. A serial-in, parallel-out register accepts serial input and provides its stored bits simultaneously at the outputs.

Block Diagram:



Code:

testbench.sv

```
module tb_SISO;
  reg clk, reset, serial_in;
  wire serial_out;
  SISO uut (.clk(clk), .reset(reset), .serial_in(serial_in),
.serial_out(serial_out));

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_SISO);
    clk = 0; reset = 1; serial_in = 0; #10;
    reset = 0;
    serial_in = 1; #10;
    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;
    serial_in = 0; #40;

    $finish;
  end
endmodule
```

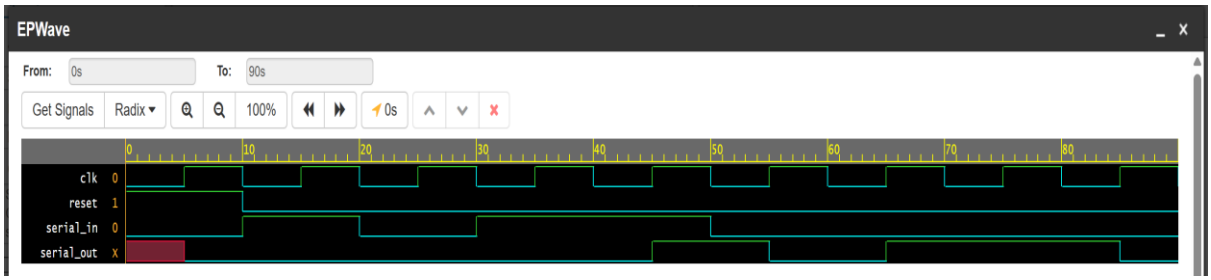
design.sv

```
module SISO(input clk, reset, serial_in, output serial_out);
  reg [3:0] Q; // Internal 4-bit memory

  always @(posedge clk) begin
    if (reset)
      Q <= 4'b0000;
    else
      Q <= {Q[2:0], serial_in}; // Shift bits left, insert new bit at
the end
  end

  assign serial_out = Q[3];
endmodule
```

Waveform:

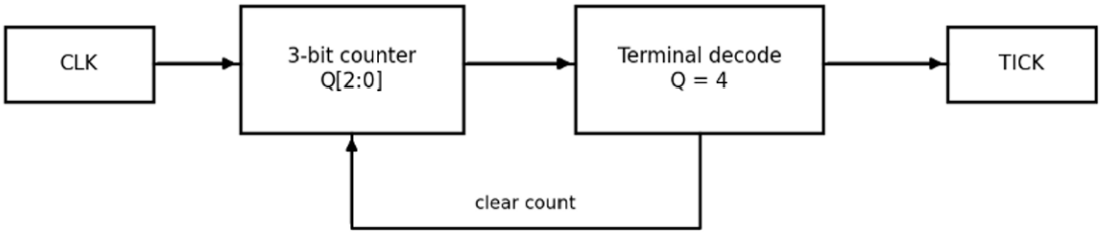


The waveform illustrates the operation of a 4-bit Serial-In Serial-Out (SISO) shift register, beginning with a high reset signal that forces the serial out line to an initial state of 0. Once the reset drops low at 10s, a sequence of binary values (1, 0, 1, 1) is pushed into the serial in pin at each positive edge of the clock (clk). Because the internal memory holds exactly four bits, it takes four full clock cycles for any single bit to travel from the input to the output. This inherent 4-cycle delay is clearly demonstrated in the waveform, as the very first '1' entered serial in at the 15s mark finally emerges on the serial outline at the 45s mark.

10. **Divide-by-N-Counter:**

A divide-by-N counter counts a fixed number of clock cycles before returning to its initial state. In the modulo-5 example, it generates a tick pulse once every five clock cycle.

Block Diagram:



Code:

```
// Code your testbench here
// or browse Examples
module tb;
reg clk,rst;
wire tick;

divide_n dut(clk,rst,tick);
always #5 clk=~clk;

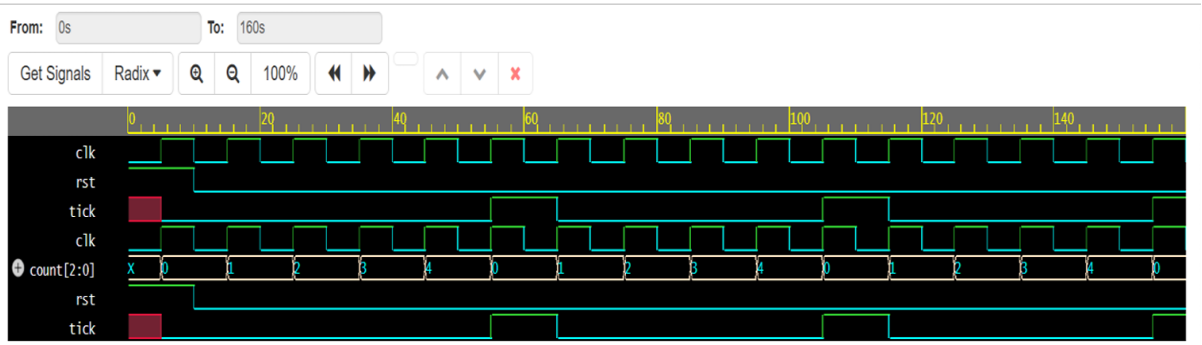
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);

    clk=0; rst=1;
    #10 rst=0;
    #150;
    $finish;
end
endmodule
```

```
// Code your design here
module divide_n(
    input clk,rst,
    output reg tick
);
reg [2:0] count;

always @(posedge clk) begin
    if(rst) begin
        count <= 0;
        tick <= 0;
    end
    else if(count == 4) begin
        count <= 0;
        tick <= 1;
    end
    else begin
        count <= count + 1;
        tick <= 0;
```

Waveform:



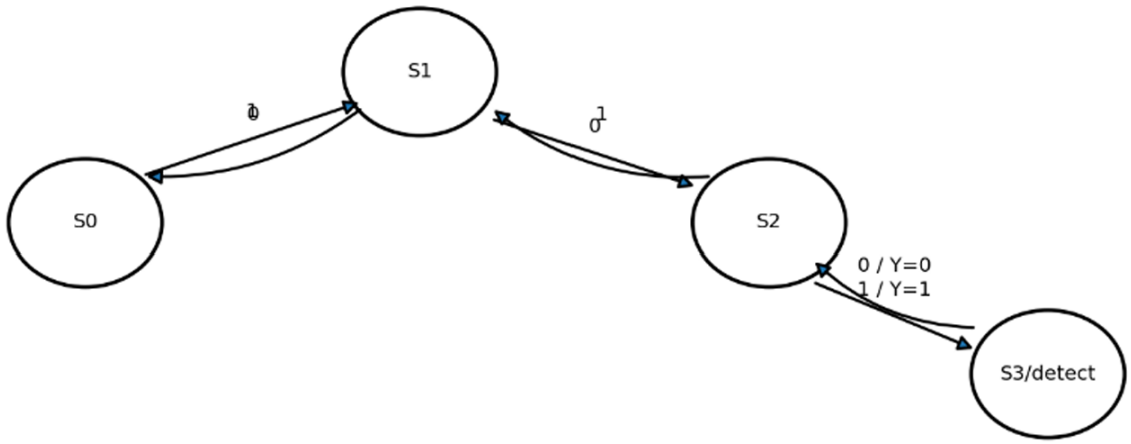
The waveform shows the clock, reset, internal counter value and tick output of the divided-by 5 circuit. Initially, reset clears the counter and sets the tick output to LOW. After reset is released, the counter progresses from zero to four and returns to zero after reaching the terminal count. The tick output becomes HIGH for one clock cycle every five-clock cycle, verifying the divide-by-5 counting operation.

11. **Mealy and Moore FSMs:**

Mealy Finite State Machine (FSM):

A Mealy FSM is a sequential circuit whose output depends on both the current state and the current input. In the 101 sequence detector, the output becomes HIGH when the final input bit completes the sequence.

Block Diagram:



Mealy output is labelled on transitions as input / output.

Code:

```
module tb_Mealy;
  reg clk, reset, din;
  wire y;
  MealyFSM uut (.clk(clk), .reset(reset), .din(din), .y(y));
  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_Mealy);
    clk = 0; reset = 1; din = 0; #10;
    reset = 0;

    // Apply input sequence: 1 -> 0 -> 1 -> 0 -> 1
    // The sequence "101" will overlap and trigger the output twice
    din = 1; #10;
    din = 0; #10;
    din = 1; #10; // Output 'y' will spike to 1 here
    din = 0; #10;
    din = 1; #10; // Output 'y' will spike to 1 again here

    din = 0; #20;

    $finish;
  end
```

```
module MealyFSM (input clk, reset, din, output reg y);
  reg [1:0] state, next_state;
  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
  always @(posedge clk) begin
    if (reset)
      state <= S0;
    else
      state <= next_state;
  end
  always @(*) begin
    y = 0;
    next_state = state;

    case (state)
      S0: begin
        if (din == 1) next_state = S1;
        else next_state = S0;
      end
      S1: begin
        if (din == 0) next_state = S2;
        else next_state = S1;
      end
      S2: begin
        if (din == 1) begin
```

Full Code:

```
module MealyFSM (input clk, reset, din, output reg y);
  reg [1:0] state, next_state;
  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
  always @(posedge clk) begin
    if (reset)
      state <= S0;
    else
      state <= next_state;
  end
  always @(*) begin
    y = 0;
    next_state = state;



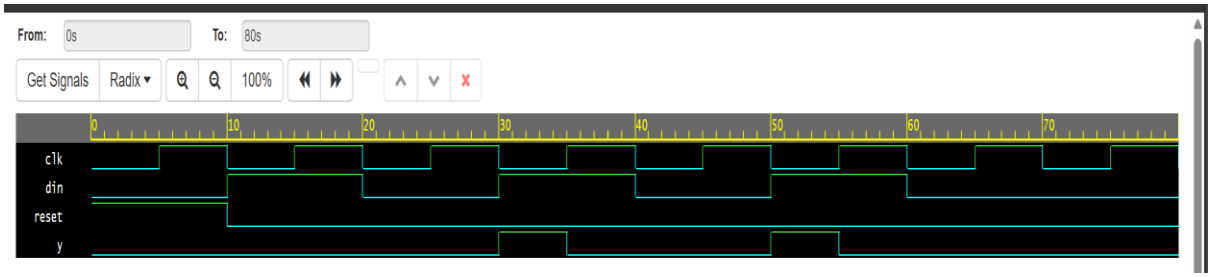
case (state)
      S0: begin
        if (din == 1) next_state = S1;
        else next_state = S0;
      end
      S1: begin
        if (din == 0) next_state = S2;
        else next_state = S1;
      end
      S2: begin
        if (din == 1) begin
          next_state = S1;
          y = 1; // Mealy Output: Becomes 1 immediately when input is 1
```

```
        end else begin
          next_state = S0;
        end
      end
    endcase
  end
endmodule
```

Testbench:

```
module tb_Mealy;
  reg clk, reset, din;
  wire y;
  MealyFSM uut (.clk(clk), .reset(reset), .din(din), .y(y));
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_Mealy);
    clk = 0; reset = 1; din = 0; #10;
    reset = 0;
    // Apply input sequence: 1 -> 0 -> 1 -> 0 -> 1
    din = 1; #10;
    din = 0; #10;
    din = 1; #10; // Output 'y' will spike to 1 here
    din = 0; #10;
    din = 1; #10; // Output 'y' will spike to 1 again here
    din = 0; #20;
    $finish;
  end
endmodule
```

Waveform:

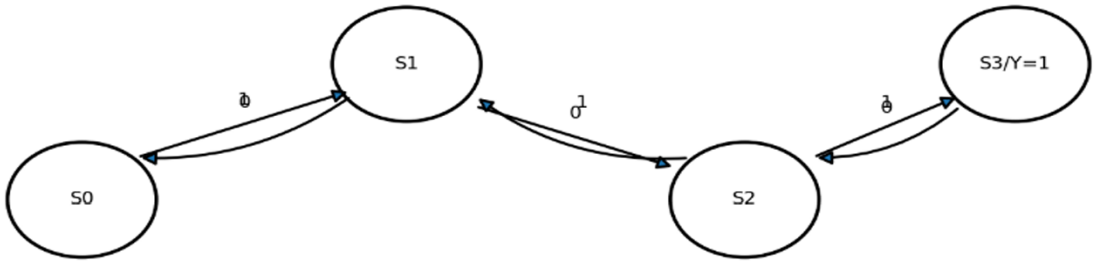


Here, beginning with a high reset signal that holds the output y at zero. After the reset drops, the data input din receives a pattern of 1s and 0s, prompting the machine to update its internal state at each positive clock edge. Crucially, at the 30-second and 50-second marks, the moment din transitions to 1 to complete the "101" sequence, the output y instantly spikes to 1 before the next clock pulse. This immediate reaction perfectly demonstrates the defining characteristic of a Mealy machine, where the output changes instantly based on the current inputs rather than waiting for the clock.

Moore Finite State Machine (FSM):

A Moore FSM is a sequential circuit whose output depends only on its current state. In the 101 sequence detector, a dedicated detection state produces HIGH output.

Block Diagram:



Moore output: Y = 1 only in state S3; output is state-decoded.

Code:

testbench.sv

```
module tb_Moore;
  reg clk, reset, in_bit;
  wire out_bit;

  MooreFSM uut (.clk(clk), .reset(reset), .in_bit(in_bit),
.out_bit(out_bit));

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd"); $dumpvars(1, tb_Moore);
    clk = 0; reset = 1; in_bit = 0; #10;
    reset = 0;
    in_bit = 1; #10; // Change state
    in_bit = 0; #20; // Hold state
    in_bit = 1; #10; // Change state
    $finish;
  end
endmodule
```

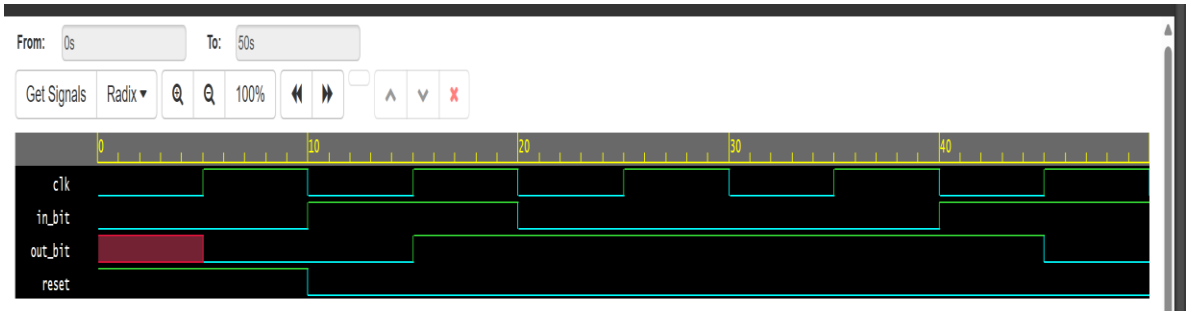
design.sv

```
module MooreFSM(input clk, reset, in_bit, output reg out_bit);
  reg state;

  always @(posedge clk) begin
    if (reset) state = 0;
    else if (in_bit == 1) state = ~state;
  end

  always @(*) begin
    out_bit = state;
  end
endmodule
```

Waveform:

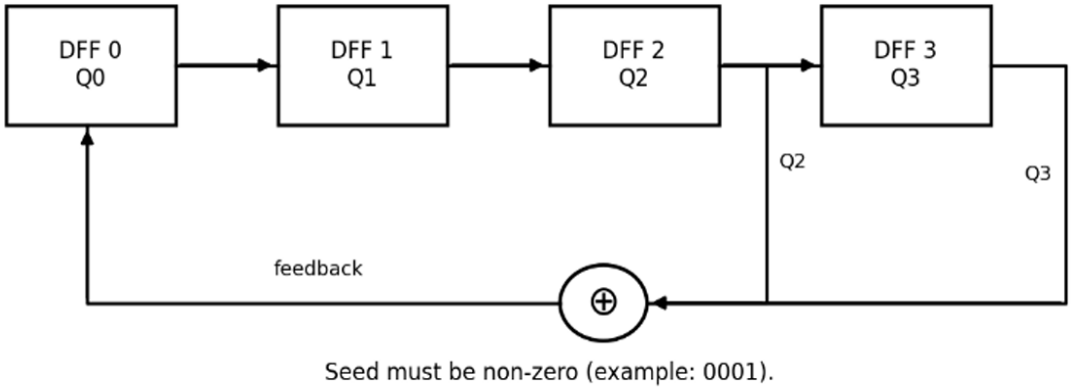


Moore finite state machine functioning as a basic toggle, starting with a high reset signal that holds the out_bit at zero. Once the reset drops low at the 10-second mark, the machine evaluates the in_bit input at every positive edge of the clock (clk). When a positive clock edge occurs while in_bit is high, seen at 15 seconds and 45 seconds, the out_bit successfully toggles its state. Because this is a Moore machine, the out_bit strictly updates only on these clock edges rather than reacting the moment the in_bit changes, ensuring perfect synchronization with the clock.

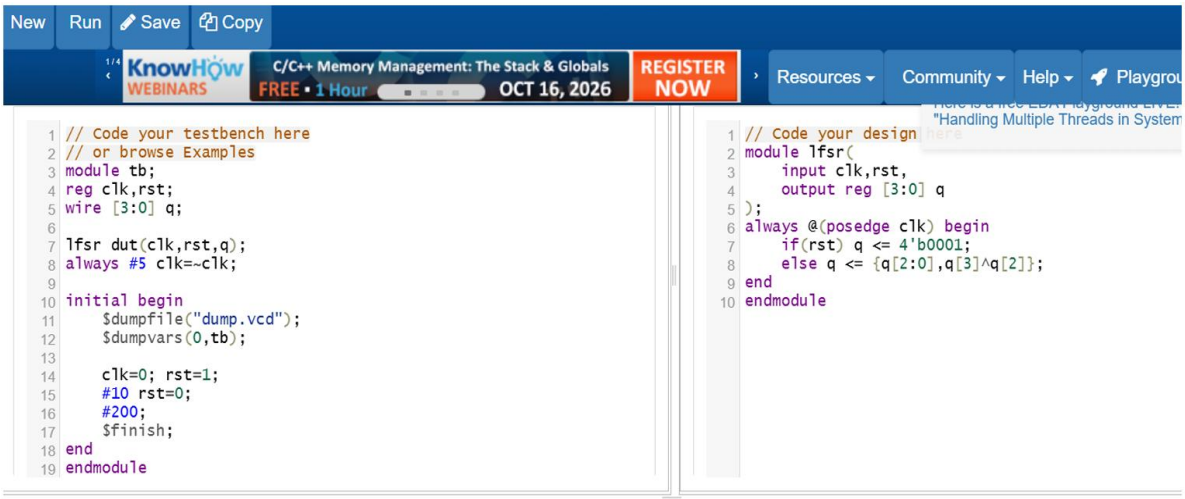
12. Random Sequence Generator:

A linear-feedback shift register (LFSR) generates a pseudo-random sequence using flip-flops and XOR feedback logic. A non-zero initial seed is required to prevent the register from remaining permanently in the all-zero state.

Block Diagram:



Code:



```
// Code your testbench here
// or browse Examples
module tb;
reg clk,rst;
wire [3:0] q;

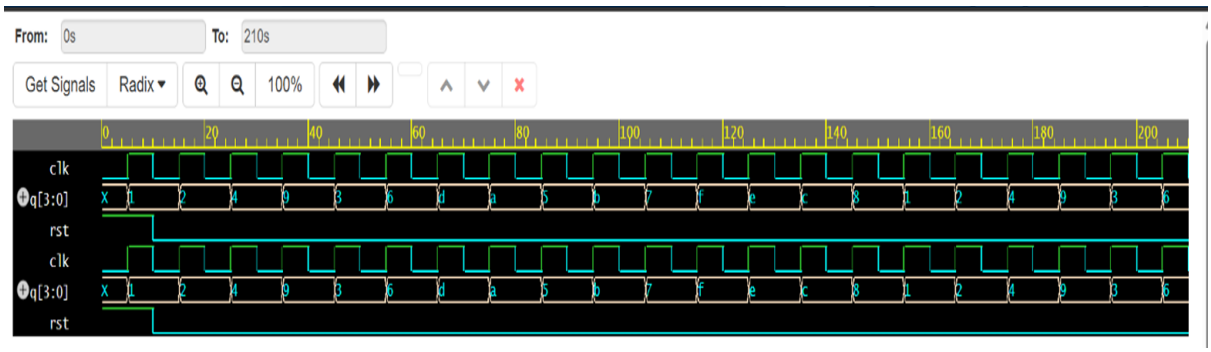
lfsr dut(clk,rst,q);
always #5 clk=~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);

    clk=0; rst=1;
    #10 rst=0;
    #200;
    $finish;
end
endmodule
```

```
// Code your design here
module lfsr(
    input clk,rst,
    output reg [3:0] q
);
always @(posedge clk) begin
    if(rst) q <= 4'b0001;
    else q <= {q[2:0],q[3]^q[2]};
end
endmodule
```

Waveform:



The waveform shows the clock, reset and four-bit output of the linear-feedback shift register. Initially, reset loads the non-zero seed 0001 into the register. On every rising clock edge, the stored bits shift and the XOR feedback generates a new bit, producing a sequence of binary states. The waveform verifies the generation of a repeating pseudo-random sequence using the LFSR circuit.

Conclusion:

The respective Verilog HDL programs are successfully designed and simulated, and their output waveforms are observed and verified.